A Modelling Framework for Scalable Near-Duplicate Detection Using Random Projections and Locality-Sensitive Hashing

Ankit Gangwar (MM25D950)
Puneet (ID25S027)
Tanmoy Ghosh (MA25M026)
Rahul Ghosh (MA25M021)

Project Objectives

- High-dimensional datasets suffer from:
  - Curse of dimensionality
  - Distance concentration
  - High computational cost
- Traditional clustering methods: Scale poorly, Sensitive to dimensionality
- Goal:
  - Use Johnson–Lindenstrauss (JL) lemma for dimensionality reduction
  - Use Locality Sensitive Hashing (LSH) for probabilistic similarity modeling
- Build a scalable framework for:
  - Approximate nearest neighbor search
  - Near-duplicate detection
  - Sublinear similarity retrieval

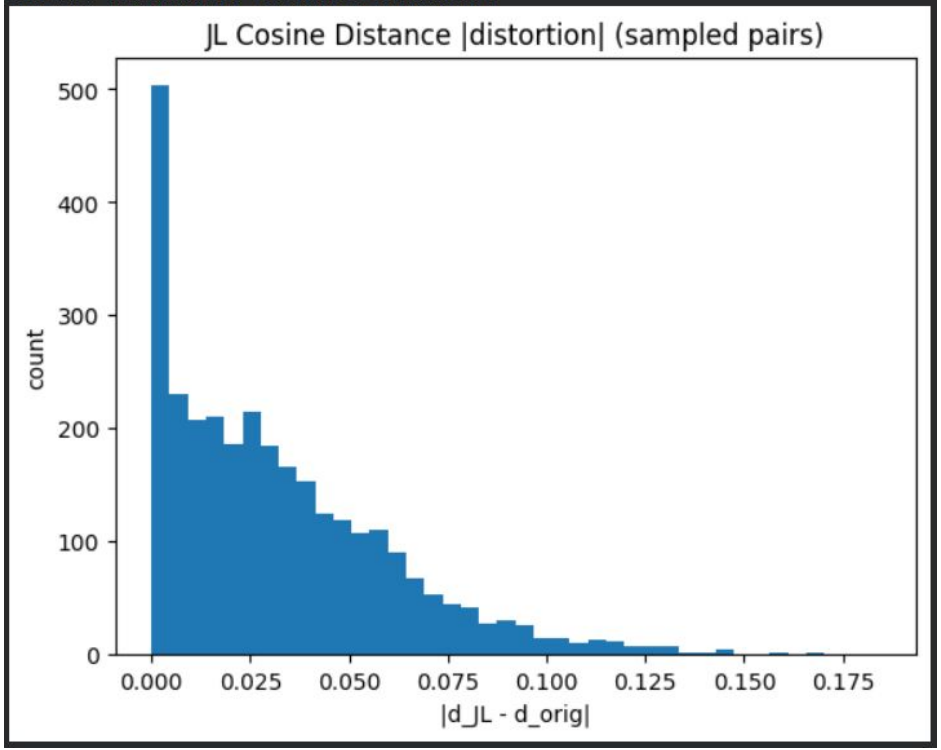
Experiment 1: 20 Newsgroups (Text)

- Dataset: 4 categories from 20 Newsgroups
- Preprocessing:
  - TF-IDF (30k features)
  - L2 normalization
- Dimensionality reduction:
  - Sparse Random Projection (JL) → 512 dimensions
- Methods compared:
  - KMeans
  - HDBSCAN
  - Custom JL + Random Hyperplane LSH

Johnson–Lindenstrauss Validation

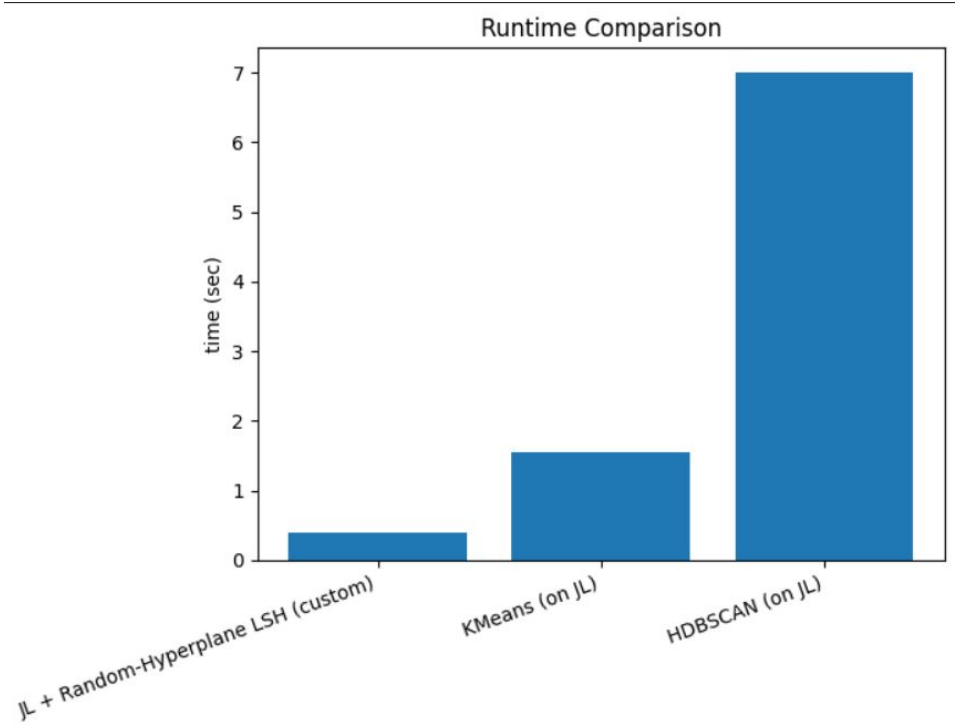
- Mean absolute cosine distortion: 0.0325
- 95% distortion bound: 0.088
- Majority of pairs preserved within < 0.05 distortion
- Confirms theoretical guarantee:
  - Random projection preserves pairwise distances approximately

```
Mean abs distortion: 0.03254999010743124
95% abs distortion: 0.08829454226806874
```



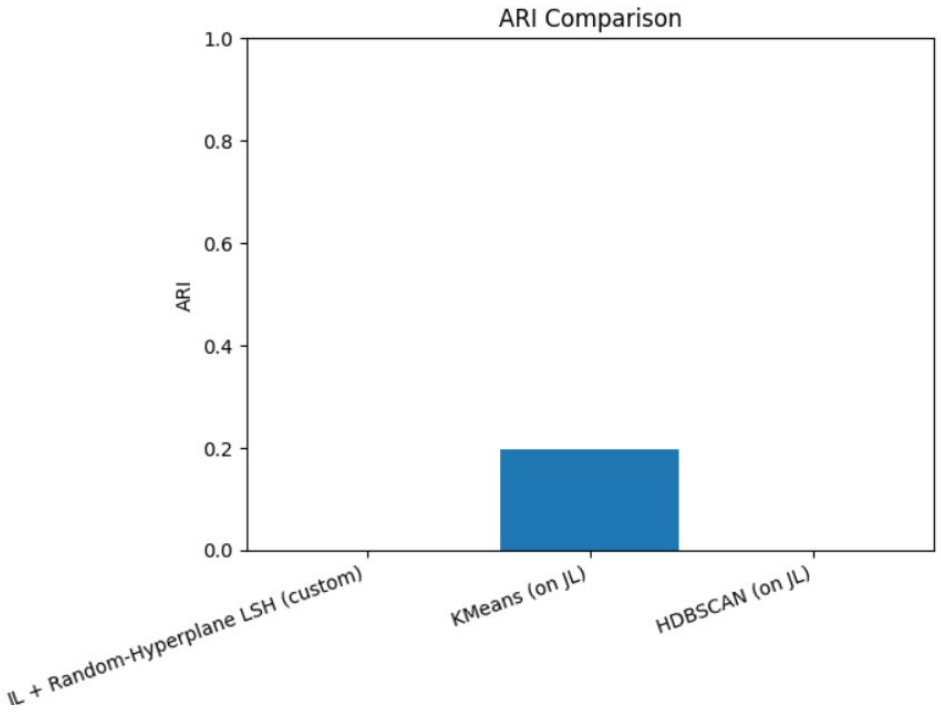
Runtime Analysis

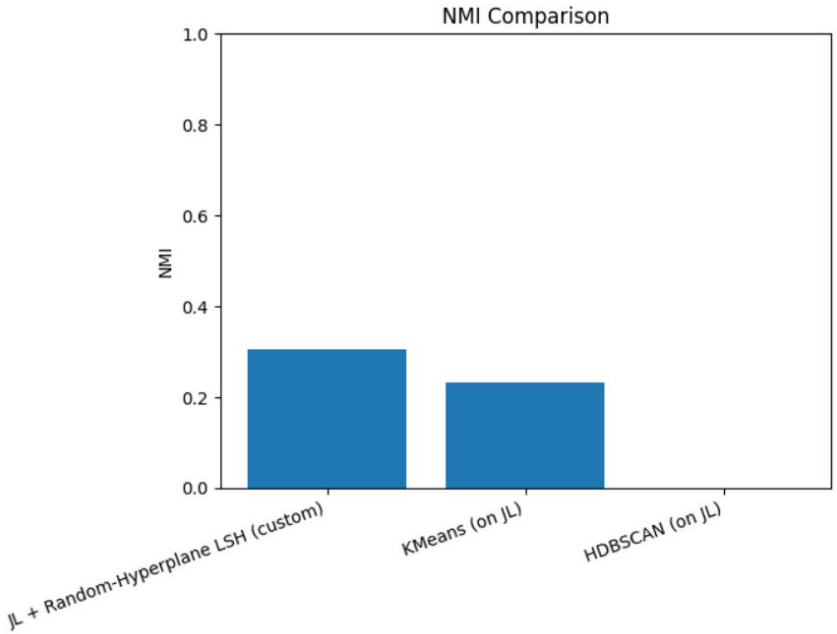
- Runtime
  - JL+LSH - 0.4 seconds
  - KMeans - 1.54 seconds
  - HDBSCAN - 7 seconds
- Key Insights -
  - LSH is:
    - ~4× faster than KMeans
    - ~17× faster than HDBSCAN
  - Sublinear candidate generation via hashing
- This demonstrates the scalability advantage of probabilistic hashing.

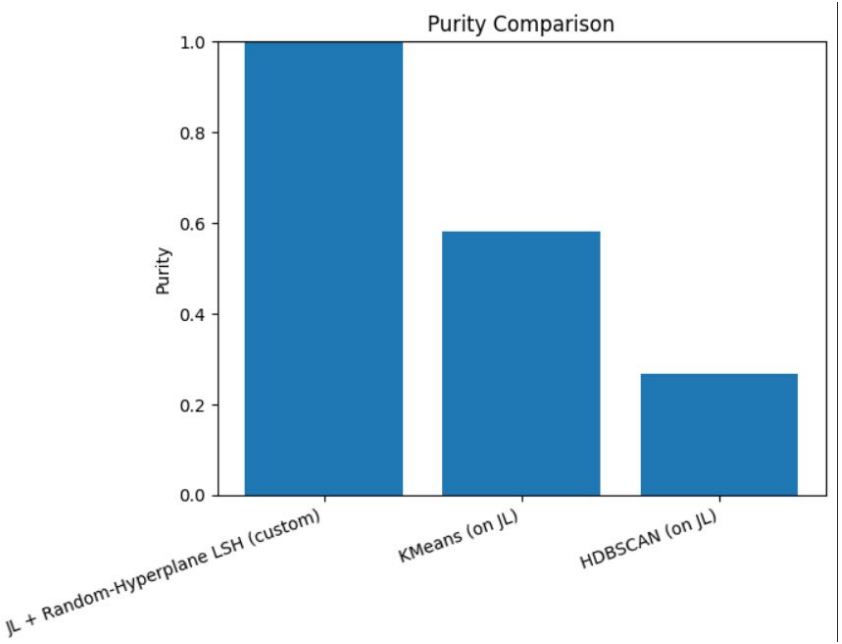


Clustering Evaluation Metrics

- Evaluation Consolidated Table -
- Key Analysis
  - KMeans
    - Best ARI (true partitioning)
  - LSH
    - High NMI (good information grouping)
    - Perfect purity due to small connected components
  - HDBSCAN
    - Failed to find stable clusters



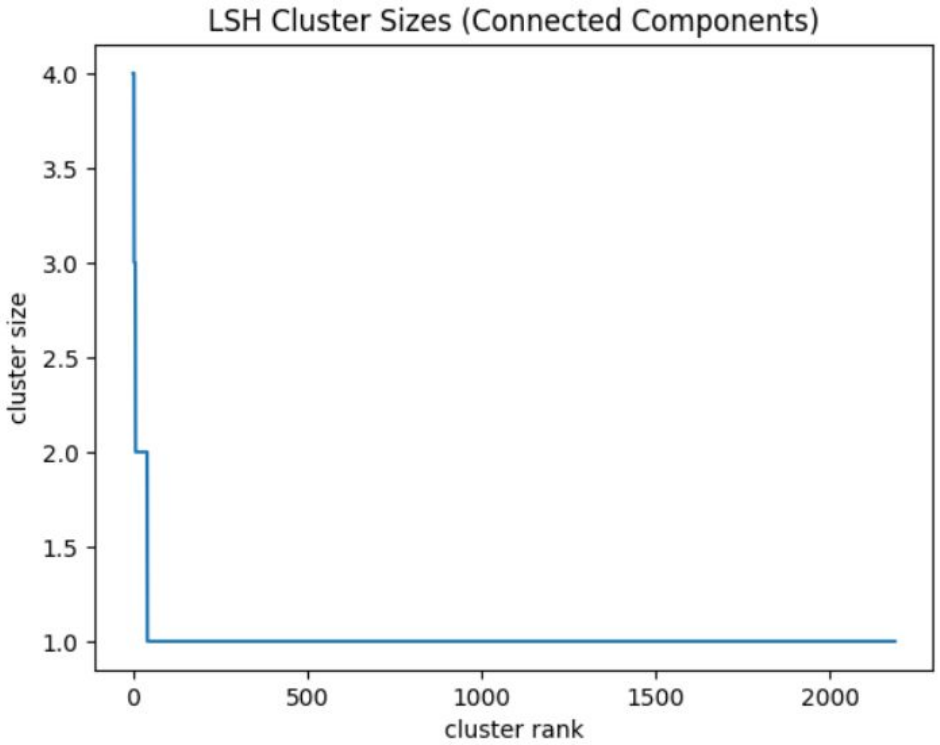

NMI Comparison
NMI
1.0
0.8
0.6
0.4
0.2
0.0
JL + Random-Hyperplane LSH (custom)
KMeans (on JL)
HDBSCAN (on JL)


Purity Comparison
Purity
1.0
0.8
0.6
0.4
0.2
0.0
JL + Random-Hyperplane LSH (custom)
KMeans (on JL)
HDBSCAN (on JL)

	method	time_sec	ARI	NMI	Purity	Silhouette	n_clusters
2	JL + Random-Hyperplane LSH (custom)	0.404925	0.000152	0.304896	1.000000	0.018226	2188
0	KMeans (on JL)	1.542790	0.197328	0.231598	0.581956	0.003431	4
1	HDBSCAN (on JL)	7.003872	0.000000	0.000000	0.266637	NaN	0

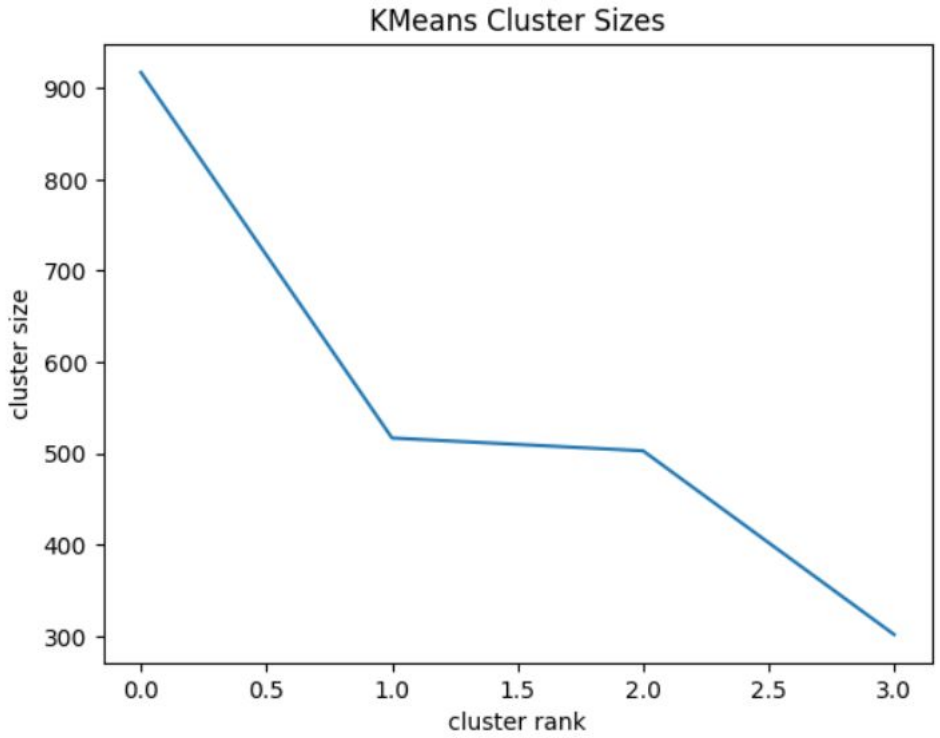
Interpretation of LSH Clustering Behavior

- LSH forms many small connected components (2188 clusters)
- Designed for:
  - Neighbor retrieval
  - Duplicate detection
- Not designed for:
  - Global partitioning into fixed k clusters
- High purity occurs because:
  - Small components rarely mix classes
- LSH is a retrieval structure, not a global clustering algorithm.



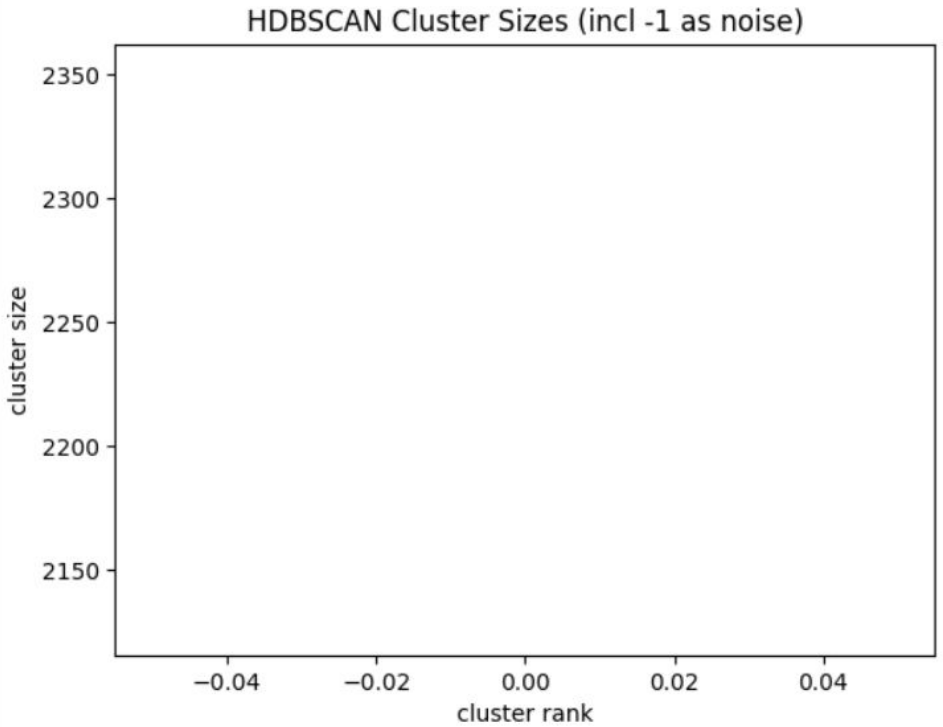
Cluster Size Distribution

- KMeans
  - 4 balanced clusters
  - Global partitioning
- LSH
  - Many small components
  - Fine-grained similarity structure
- KMeans forces structure. LSH discovers local similarity graph structure.



High-Dimensional Density Clustering Challenge

- HDBSCAN found 0 meaningful clusters
- Almost entire dataset treated as noise
- This demonstrates that -
  - Density-based methods struggle in high dimensions
  - Distance concentration problem



Near-Duplicate Retrieval Experiment

- Synthetic duplicates created via word corruption
- LSH recall: 80.8%
- Runtime: 0.59 sec
- This demonstrates that -
  - Collision probability reflects cosine similarity
  - Efficient approximate neighbor retrieval
- This is where LSH shines. It retrieves near duplicates extremely efficiently.

```
Duplicate recall (LSH components): 0.8083333333333333
Time (LSH on augmented): 0.5892111449999788
```

Conclusion

- JL effectively preserves geometry
- LSH achieves major runtime improvement
- KMeans best for global partitioning
- LSH best for scalable similarity retrieval
- Probabilistic modeling provides scalable alternative to exact clustering